

Chapter 15

Native Multi-Objective Mathematical Programming Software

Xavier Gandibleux

Abstract Multi-objective mathematical programming is a fundamental paradigm for decision-making problems involving conflicting criteria. However, only a limited set of software provide *native* support for multi-objective optimisation, meaning that multiple objective functions are treated as first-class entities by the modeler or the solver itself. This chapter provides a comprehensive overview of optimisation modelers and optimisation solvers offering native multi-objective capabilities, excluding software that explicitly exploit preference information, and software based on heuristics and metaheuristics solution techniques. Therefore, aspects of modelers and solvers related to nonlinear optimisation are not covered. An illustrative bi-objective integer linear programming example is modeled and solved using relevant software available in 2026, highlighting both syntactic and semantic differences, and to demonstrate different concrete modeling and solution workflows.

Key words: Modelers, Algebraic Modeling Languages, Solver-Oriented Interfaces, Solvers, Generic MILP Solvers, Specific MOMP Solvers, Meta-Solvers

Xavier Gandibleux
Nantes Université, Faculty of Sciences and Technics, 2 Rue de la Houssinière BP 92208,
F44322 Nantes, France. e-mail: Xavier.Gandibleux@univ-nantes.fr

15.1 Introduction

Many real-world optimisation problems involve multiple objectives. Such problems arise in a wide range of application domains, including energy planning, transportation, supply chain management, logistics, finance, engineering design, public policy, or again economics. In these settings, objective functions describing cost, risk, efficiency, robustness, and environmental impact may be stated and are often inherently conflicting. As a result, decision-makers are rarely interested in a single optimal solution; instead, they seek to understand the trade-offs between competing goals. This makes single-objective formulations insufficient to adequately capture the decision-making context.

Multi-objective mathematical programming (MOMP) provides a formal framework for addressing such problems. In accordance with the definitions and notations stated in Chapter 1 of the book, also found in [22], a general multi-objective optimisation problem can be written as:

$$\min z(x) = (z_1(x), \dots, z_p(x)) \quad s.t. \quad x \in \mathcal{X},$$

where $\mathcal{X} \subseteq \mathbb{R}^n$ is the feasible set. Unlike single-objective optimisation, MOMP typically does not yield a unique optimal solution. Rather, it computes a set of efficient solutions, each representing a different trade-off among the objectives. The definitions corresponding to a *complete set of efficient solutions* $\mathcal{X}_E$, to the *set of nondominated points* $\mathcal{Z}_N$, as well the definitions qualifying the *supported and non-supported nondominated points* leading to the two sets of points, $\mathcal{Z}_{SN}$ and $\mathcal{Z}_{NN}$, corresponding to them respectively, are also used throughout the chapter.

15.1.1 Native Multi-Objective Mathematical Programming Software

Despite strong theoretical foundations, the practical adoption of multi-objective optimisation remains highly dependent on software capabilities. In particular, a significant distinction exists between tools that support multi-objective optimisation *natively* and those that rely e.g. on user-defined scalarization. This chapter focuses exclusively on the former category. An optimisation software is said to provide *native multi-objective support* if

- multiple objective functions can be explicitly declared in the modeling language,
- objectives are internally handled by the modeling language or the solver,
- no manual scalarization or reformulation is required,
- the semantics of multi-objective optimisation are preserved throughout the solution process.

This definition excludes tools where multi-objective behavior is emulated solely at the user level.

Solution methods for MOMP are commonly classified according to how decision-maker preferences are articulated, namely *a priori*, *interactive*, and *a posteriori* approaches. In this chapter, we restrict attention to *a posteriori* methods, in which the optimisation process computes efficient solutions independently of explicit preference information. Consequently, software tools based on the interactive approach, such as NIMBUS (Nondifferentiable Interactive Multiobjective BUNDLE-based optimisation System) [45], DESDEO (Decision Support for computationally Demanding optimisation problems) [46], or iMOLPe (Interactive Multi-Objective Linear Programming explorer) [6], are not considered. Likewise, we consider only software tools that implement deterministic optimisation techniques grounded in mathematical programming. Approaches based on heuristics and metaheuristics optimisation paradigms, including evolutionary frameworks such as jMetal (a framework for multi-objective optimisation with metaheuristics) [7, 21], MetaJul (implementation of metaheuristics in the Julia programming language) [48], Pymoo (a Python framework for single- and multi-objective optimisation algorithms) [9] or MOEA framework (an open source Java library for developing and experimenting with multiobjective evolutionary algorithms) [31], are therefore also excluded from the scope of this chapter.

15.1.2 Timeline

The concept of native multi-objective optimisation software has a relatively short history. The seminal theoretical works that led to the very first software programs concern the multicriteria simplex algorithm [53, 54] and the Benson's outer approximation algorithm [8]. They have led, respectively, to two software: ADBASE [51] implemented in Fortran and introduced in the 1970s for the former, and BENSOLVE [42] implemented in C and introduced in 2012 for the latter.

It was not until around 2016–2017 that a significant breakthrough occurred. Commercial solvers such as IBM ILOG CPLEX optimisation [36] and Gurobi optimisation [30] introduced native multi-objective capabilities, while open-source initiatives like PolySCIP [11], vOptSolver [25, 27], and Inner [15] appeared with advanced support for vector-valued objectives and computation of the nondominated set.

This period was inspiring and motivating for the development of native multi-objective mathematical programming software. This marked a turning point for this class of optimisation software, which today also includes GAMS [24], AMPL [3], AIMMS [2], MATLAB [44], FICO Xpress optimisation [23], Hexaly [34], and COPT (Cardinal Optimizer) [14] for commercial software, CVXPY [16], HiGHS [35], and PaMILO [10] for open-source software.

15.1.3 Classification

The software discussed in this chapter belongs to at least one of the following two categories.

1. *Modelers.*

Modelers are software tools devoted to express mathematical optimisation problems in a form that closely mirrors their algebraic formulation. They provide a dedicated syntax that allows users to define decision variables, parameters, constraints, and objective functions using mathematical notations. Modelers facilitate the separation between the model definition and the data, which enhances readability, reusability, and maintainability. Behind this general term stand two clearly identified groups of modeling tools: (a) Algebraic Modeling Languages, and (b) Solver-Oriented Interfaces. The two groups differ primarily in their level of abstraction, solver independence, and intended use.

a. *Algebraic Modeling Language.*

An *Algebraic Modeling Language* (AML) is a high-level programming language specifically designed for mathematical optimisation, with an emphasis on declarative modeling and independent of any particular solver. The design of AMLs aims to bridge the gap between the mathematical model and the underlying optimisation engine, facilitating clear, maintainable, and solver-independent problem representations of mathematical programs. Therefore, AMLs favor portability, clarity, and the ability to experiment across multiple solvers with minimal modification. Once a model is specified, an AML typically generates an internal representation or standardized exchange formats that can be passed to different optimisation solvers without modifying the model itself. This high level of abstraction makes AMLs particularly well suited for research, prototyping, rapid experimentation, and comparative studies across solvers.

GAMS [24], AMPL [3], AIMMS [2], OPL [36], Mosel [23], GMP [29], ZIMPL [39], JuMP [20, 43], Pyomo [33] are among the most important AMLs.

Some AMLs are close to programming languages, such as Julia for JuMP or Python for Pyomo, allowing users to leverage familiar programming constructs, data structures, and libraries alongside mathematical modeling capabilities. Others are expressed in proprietary languages, such as GAMS, AMPL, AIMMS, OPL, Mosel, or ZIMPL.

b. *Solver-Oriented Interfaces.*

In contrast, a *Solver-Oriented Interface* (SOI) is tightly coupled to a specific optimisation solver and is primarily intended to provide programmatic control and access to solver-specific features. These interfaces are often implemented as *Application Programming Interfaces* (API) or embedded *Domain-Specific Languages* (DSL) within a host programming language such as C/C++, Java, or Python. This close integration enables fine-grained

tuning and control over solver parameters, algorithmic options, and advanced solver-specific features, making these interfaces particularly suitable for applications requiring deep customization of the solution process, but with reduced portability. Therefore, Solver-Oriented Interfaces are well suited for integration into larger software systems.

MATLAB [44], API C++ Gurobi [30], highspy [35], Hexaly Modeler [34] are examples belonging into this category.

2. Solvers.

An *optimisation solver* is a specialized software engine designed to find solutions to mathematical programming problems by systematically exploring the feasible region defined by constraints and optimizing one or more objective functions.

Behind this second general term two categories are distinguished here: (a) generic MILP solvers, and (b) specific MOMP solvers.

a. Generic MILP solvers.

Generic MILP solvers implement numerical algorithms tailored to problem classes such as linear, mixed-integer, nonlinear, convex, or multi-objective programs. They handle the computational complexity of large-scale models, provide solution methods such as simplex, branch-and-bound, constraint programming, or interior-point techniques, and may support advanced features like computation of some efficient solutions with e.g. a weighted-sum optimisation, or hierarchical optimisation. Gurobi [30], CPLEX [36], Xpress [23], MOSEK [47], Hexaly [34], COPT [14], GLPK [28], SCIP [1], HiGHS [35], CBC/CLP [13] are among the most important LP/MILP solvers.

b. Specific MOMP solvers.

Specific solvers are specialized optimisation engines designed to directly handle problems with multiple objectives. On the one hand, and unlike AMLs and SOIs, several MOMP solvers have adopted specific file formats, dedicated to a given solver, to define objectives, constraints, and variables; these formats are typically limited to the solver's ecosystem. Examples include BENSOLVE [42], INNER [15], and PaMILO [10]. On the other hand, there is the example of the vOptSpecific [27] solver, which has chosen to implement a Julia API guided by the optimisation problems that it can handle, and to use the MOP format, which is an extension of the multi-objective MPS format introduced by PolySCIP, for persistent storage needs.

Remarks :

- Modern optimisation solvers are almost exclusively implemented in C or C++ in order to meet stringent performance, memory, and numerical efficiency requirements. However, the emergence of high-level/high-performance languages such as Julia, which combine execution speeds comparable to C/C++ with expressive syntax and high-level abstractions comparable e.g. to MATLAB, has recently changed this landscape. As a result, several recent optimisation solvers

and algorithmic frameworks are now implemented entirely in Julia, benefiting all advantages provided by the language, and escape from the *two-language problem*.

- optimisation solvers are typically interfaced with modeling languages to enable efficient and accurate solution of complex mathematical problems. In order to facilitate communication from/to modelers and solvers, such as loading and storing of mathematical models from/to a file, various formats exist. Among these, the MPS and LP formats are two historical encoding formats—independent of the solver—that are still in use today. They serve e.g. as standardized input and output interfaces between a modeler and an optimisation solver for single objective mathematical problems.
 - The MPS format provides an atomized textual description of linear and mixed-integer programs. Although outdated, it remains widely present in optimisation software for its technical simplicity and backward compatibility.
 - The LP format gives a more human-readable algebraic description of the represented problem, facilitating inspection and debugging.

In both cases, these formats abstract away high-level modeling constructs while preserving the mathematical structure required by the solver, thereby enabling interoperability between AMLs/SOIs and a wide range of optimisation engines. It should be noted that there have been several attempts to propose a format – most extending existing single objectives – to describe a mathematical problem with multiple objectives, but none of them has become widely accepted.

In the following, we will limit ourselves to modelers and solvers with native multi-objective support. Also we focus exclusively on linear optimisation problems, including linear programs (LP) and mixed-integer linear programs (MIP).

15.1.4 Illustrative Example

Consider the following bi-objective integer linear programming problem (Figures 15.1 and 15.2):

$$\max z_1 = x_1 + x_2 \quad (0.1)$$

$$\min z_2 = x_1 + 3x_2 \quad (0.2)$$

s.t.

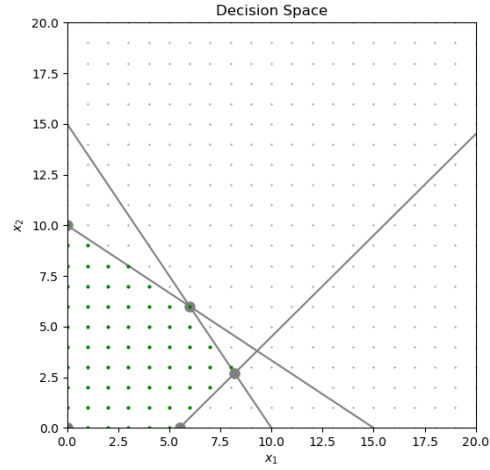
$$2x_1 + 3x_2 \leq 30 \quad (1)$$

$$3x_1 + 2x_2 \leq 30 \quad (2)$$

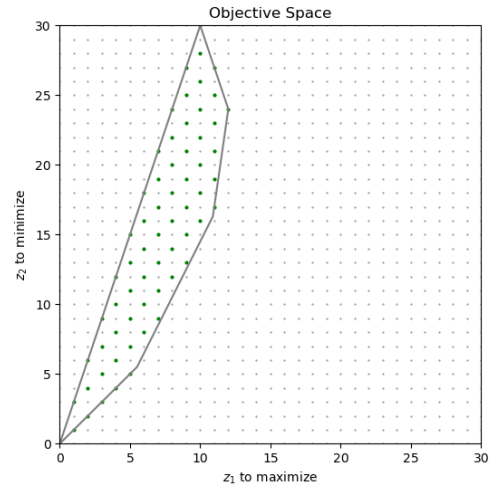
$$x_1 - x_2 \leq 5.5 \quad (3)$$

$$x_1, x_2 \in \mathbb{N}$$

The two objectives represent conflicting goals: maximizing total output (objective (0.1)) while minimizing a cost-related expression (objective (0.2)).



(a) The decision space $\mathbb{R}^2$, and the integer feasible region $\mathcal{X}$ of the bi-objective problem (green dots).

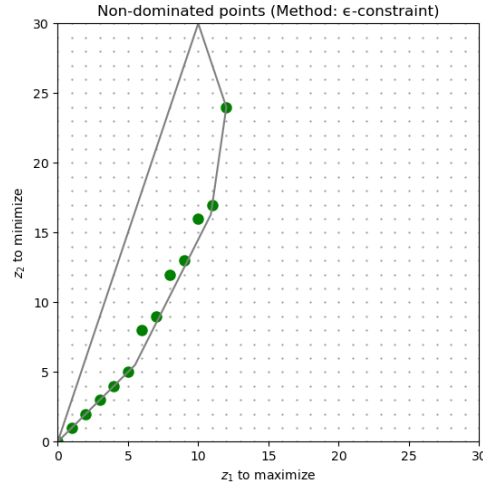


(b) The objective space $\mathbb{R}^2$, and the image $\mathcal{Z} = z(\mathcal{X})$ of integer feasible solutions (green dots).

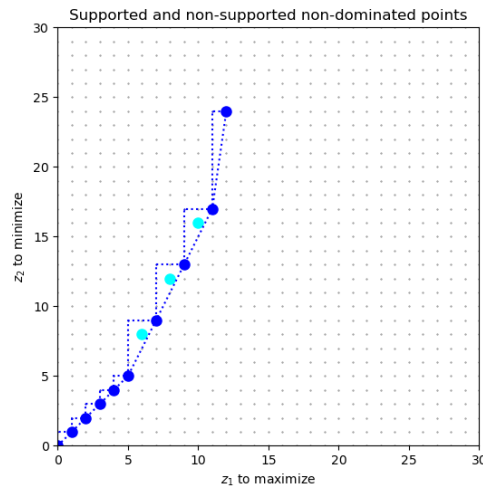
Fig. 15.1: Decision and objective spaces of the bi-objective illustrative example.

15.2 Modeling and Solving with Software Supporting Native Multi-Objective Optimisation

All modelers presented in this section allow the explicit declaration of multiple objective functions at the modeling level. Also, all solvers presented provide built-in



(a) $\mathcal{Z}_N$, the set of nondominated points (large green dots).



(b) $\mathcal{Z}_{SN}$ and $\mathcal{Z}_{NN}$, the sets of supported (large dark blue dots) and nonsupported (large light blue dots) points.

Fig. 15.2: Sets of nondominated points, supported and non-supported nondominated points for the bi-objective illustrative example.

methods for multi-objective optimisation, noting that these capabilities are generally restricted to linear problem formulations only.

Common features shared by most of the software discussed are not reiterated in the remainder of this chapter; for instance, most solvers provide access through multiple programming interfaces, ranging from native declarative modeling languages specific to the software (e.g., *HXM* for Hexaly, *MOSEL* for FICO or again *OPL* for CPLEX) to Python APIs for seamless integration with Python workflows, Java APIs for enterprise applications, and C/C++ APIs for high-performance computing environments. Likewise, unless otherwise specified, the modelers and solvers presented offer built-in support for standard linear and mixed-integer programming (LP and MIP) formats.

Most of the computing methods introduced in reviewed software rely on three simple scalarization methods to compute trade-offs between conflicting objectives: (1) the hierarchical method, (2) the weighted sum method, and (3) the epsilon constraint method.

- Hierarchical method.

Objectives are ordered by the decision-maker by priority and optimized sequentially by the solver, each objective being optimized subject to the optimality of all higher-priority ones. This approach returns at most one solution, corresponding to a single compromise represented by the chosen priority sequence. Its main limitation is that it does not explore trade-offs, changing priorities may lead to a completely different solution. This method does not provide insight on the trade-offs.

- Weighted-sum method.

All objectives are aggregated into a single scalar objective using weights defined by the decision-maker. Each set of weights yields one compromise solution, and multiple runs with different weights are required to explore trade-offs. This approach is straightforward but has well-known limitations specially for MIP problems: it cannot capture non-convex regions of the nondominated set of points, and is highly sensitive to the choice and scaling of weights.

- Epsilon-constraint method.

One objective is optimized while the others are converted into constraints bounded by epsilon values. By varying these bounds, the method can generate multiple trade-offs, potentially approximating or fully enumerating the set of nondominated points by solving a series of single-objective problems. The choice of epsilon values is important, and unlike the weighted sum approach, this method can discover all trade-offs, including those in non-convex regions of the nondominated set.

Note that hierarchical method is a special case of lexicographic method, as it is limited to a predefined set of objective priorities. Also, these three methods can be regarded as a priori approaches, since in each case the decision maker must provide preference information in advance (objective priorities, weights, or constraint bounds).

Software are discussed in this section using a unified typographical style to ensure consistency and facilitate direct comparison. The formalisms used to model the illustrative problem are selected to showcase, as broadly as possible, the diversity

of modeling approaches available. The differences between the software programs are therefore expressed solely by the syntax and modeling constructs. The sequence in which the software are presented reflects, on the one hand, an increasing abstraction level of the associated modeling languages and, on the other hand, their native capabilities for multi-objective mathematical programming.

The presentation of the existing software aims to be as complete and concise as possible; we apologize for any potential omissions, especially given that the field is evolving rapidly. Also, all code snippets presented have been executed using the software mentioned; however, since the underlying languages may evolve over time, it is the reader's responsibility to adapt the code provided to the latest specifications of the corresponding software, if necessary.

15.2.1 MATLAB [44]

*MATLAB Optimisation Toolbox*¹ is a commercial software which provides built-in functions for optimisation through a Solver-Oriented Interface. MATLAB's programming flexibility and extensive numerical computing capabilities make it popular for prototyping optimisation models, particularly in engineering and scientific computing contexts where matrix operations are central to the workflow.

For multi-objective optimisation, MATLAB has primitives for setting multiple objectives and the *Global Optimisation Toolbox* of MATLAB provides dedicated functions for generating approximations of $\mathcal{Z}_N$. The two main solvers are `paretosearch` (a direct search method using pattern search) and `gamultiobj` (a controlled elitist genetic algorithm based on NSGA-II). However, MATLAB lacks the multi-objective capabilities found in almost all solvers discussed in the rest of the chapter, and provides no native support for the computation of $\mathcal{Z}_N$.

```

1 % f1 = -(x1+x2) (maximize x1+x2)
2 % f2 = x1+3*x2 (minimize x1+3*x2)
3 fun = @(x) [ -(x(1) + x(2)); x(1) + 3*x(2) ];
4
5 % Problem setup
6 nvars = 2;           % number of decision variables
7 intcon = [1 2];      % both variables must be integers
8 lb = [ 0 0];         % lower bounds
9 ub = [100 100];      % upper bounds
10
11 A = [ 2 3 ;          % constraints A*x <= b
12       3 2 ;
13       1 -1 ];
14 b = [ 30 ;
15       30 ;
16       5.5 ];
17

```

¹ <https://fr.mathworks.com/products/matlab.html>

```

18 % Options for the genetic algorithm and run
19 options = optimoptions('gamultiobj', 'Display', 'iter');
20
21 [x_gamut, fval_gamut, exitflag, output, population, scores]
22     = ...
    gamultiobj(fun, nvars, A, b, [], [], lb, ub, [], intcon
        , options);

```

Listing 15.1: Formulation with MATLAB using `gamultiobj`.

Listing 15.1 models the illustrative example introduced in Subsection 15.1.4 and uses `gamultiobj` to generate an approximation of $\mathcal{Z}_N$. The objective function is defined as a vector-valued function (line 3) with two conflicting objectives: minimizing $-(x_1 + x_2)$ (equivalent to maximizing $x_1 + x_2$) and minimizing $x_1 + 3x_2$. The problem features two decision variables x_1 and x_2 that are constrained to be integers (lines 6~7), as specified by the `intcon` parameter, and both variables have bound constraints restricted to the range $[0, 100]$ via the `lb` and `ub` parameters (lines 8~9). Additionally, three linear inequality constraints are imposed through the matrix `A` and vector `b`, representing the conditions $2x_1 + 3x_2 \leq 30$, $3x_1 + 2x_2 \leq 30$, and $x_1 - x_2 \leq 5.5$ (lines 11~16).

MATLAB is considered in this review of software, even though it yields only approximations of non-dominated sets, as it supports multi-objective modeling and remains widely used by the multicriteria decision-making community. For example, we can find repositories containing MATLAB implementations of algorithms dedicated to multi-objective optimisation, such as those developed at TU Ilmenau within the Group for Mathematical Methods in Operations Research (Germany)².

15.2.2 Gurobi Optimizer [30]

*Gurobi Optimizer*³ is a state-of-the-art commercial solver renowned for its performance and algorithmic capabilities, accessible through multiple modeling interfaces including e.g. native API in C/C++, `gurobipy` which is a SOI in Python, and a seamless integration with popular third-party AMLs such as JuMP (Julia), Pyomo (Python), AMPL, or again GAMS through solver interfaces, making it one of the most versatile optimisation platforms with extensive language support across academic and industrial environments. It is particularly suitable for applications requiring rapid, high-quality solutions to large-scale optimisation problems.

For multi-objective optimisation, Gurobi provides flexible methods that allow users to either blend multiple objectives through weighted combinations or treat them hierarchically by setting priorities, or combine both approaches. However, it does not provide built-in functionality for the computation of $\mathcal{Z}_N$.

² <https://www.tu-ilmenau.de/en/mmor/software>

³ <https://www.gurobi.com/>

```

1 #include "gurobi_c++.h"
2 #include <iostream>
3
4 int main() {
5
6     // Create Gurobi environment and model
7     GRBEnv env = GRBEnv();
8     GRBModel model = GRBModel(env);
9     model.set(GRB_StringAttr_ModelName, "
        BiObjectiveHierarchical");
10
11     // Decision variables (non-negative integers)
12     GRBVar x1 = model.addVar(0.0, GRB_INFINITY, 0.0,
        GRB_INTEGER, "x1");
13     GRBVar x2 = model.addVar(0.0, GRB_INFINITY, 0.0,
        GRB_INTEGER, "x2");
14
15     // Add constraints
16     model.addConstr(2*x1 + 3*x2 <= 30, "constr1"); // (1)
17     model.addConstr(3*x1 + 2*x2 <= 30, "constr2"); // (2)
18     model.addConstr( x1 - x2 <= 5.5, "constr3"); // (3)
19
20     // Set multi-objective mode
21     model.set(GRB_IntAttr_ModelSense, GRB_MAXIMIZE);
22
23     // Obj 1: maximize z1 = x1 + x2 (high priority = 2)
24     GRBLinExpr obj1 = x1 + x2;
25     model.setObjectiveN(
26         obj1,      // expression
27         0,         // index
28         2,         // priority (higher = optimized first)
29         1.0,       // weight (positive for maximization)
30         1e-6,      // absolute tolerance
31         0.0,       // relative tolerance
32         "z1" );    // name
33
34     // Obj 2: minimize z2 = x1 + 3*x2 (low priority = 1)
35     GRBLinExpr obj2 = x1 + 3*x2;
36     model.setObjectiveN(
37         obj2,      // expression
38         1,         // index
39         1,         // priority (lower = optimized second)
40         -1.0,      // weight (negative for minimization)
41         1e-6,      // absolute tolerance
42         0.0,       // relative tolerance
43         "z2" );    // name
44
45     // Solve the model
46     model.optimize();
47     return 0;
48 }

```

Listing 15.2: Formulation with Gurobi using the C++ API for the hierarchical optimization.

The C++ program given by Listing 15.2 implements the resolution of the illustrative example using Gurobi solver with hierarchical method (objective 1 first, objective 2 second) for which tolerance parameters can be introduced to control the permissible deviation from optimality when moving to lower-priority objectives. Indeed, it offers a fine-grained control over allowable degradation through two parameters, the absolute tolerance (`ObjNAbsTol`) and the relative tolerance (`ObjNRelTol`), enabling users to specify how much an objective can degrade when optimizing lower-priority goals.

The program begins by creating a Gurobi environment and model (lines 7~9), then defines two non-negative integer decision variables `x1`, `x2` using `addVar` with `GRB_INTEGER` type (lines 12~13). Three linear constraints are added (lines 16~18) via `addConstr` to define the feasible region $\mathcal{X}$. `setObjectiveN` allows defining multiple objectives (lines 24~42) with distinct priorities: the first objective ($z1 = x1 + x2$) is assigned priority 2 with positive weight 1.0 for maximization, while the second objective ($z2 = x1 + 3*x2$) receives priority 1 with negative weight -1.0 for minimization. In Gurobi's priority system, higher numerical priority values are optimized first, so objective `z1` (priority 2) is optimized before objective `z2` (priority 1). The `setObjectiveN` method's index parameter (0, 1) uniquely identifies each objective.

Gurobi offers other ways to express a mathematical model, including using `guroipy` (as described in the subsections on `HiGHS` and `COPT`).

15.2.3 HiGHS [35]

*HiGHS*⁴ is a high-performance free and open-source optimisation solver released under the MIT license. It is developed by the University of Edinburgh's Optimisation and Operational Research Group (Scotland). Algorithms integrated to *HiGHS* are primal/dual revised simplex, interior point, and branch-and-cut algorithms, accessible through multiple modeling interfaces. The wide range of ways in which *HiGHS* can be accessed –either through numerous modeling environments or via the various supported standard file formats– makes it one of the most accessible and license-friendly optimisation platforms, particularly well suited for academic research, open-source projects, and industrial applications requiring transparent, cost-free solver technology with performance comparable to commercial alternatives for many problem classes.

HiGHS supports multi-objective optimisation through either blending using weights or performing hierarchical optimisation. Multi-objective functionality is provided via the `addLinearObjective` method, where users specify objectives with priority values and weights. When `blend_multi_objectives` is true (default), objectives are combined according to weights and minimized jointly, while setting it to false triggers hierarchical optimisation that sequentially optimizes objectives

⁴ <https://highs.dev/>

by priority, with tolerance parameters to control allowable degradation when optimizing lower-priority objectives.

HiGHS does not provide built-in functionality for the computation of $\mathcal{L}_N$. But, owing to its modern architecture and open-source nature, HiGHS represents a strong candidate for integrating a parametric linear programming, which will provide an implementation of the bi-objective simplex algorithm.

```

1 import highspy
2
3 # Create the model
4 h = highspy.Highs()
5
6 # Variables x1 and x2 (integers, >= 0)
7 x1 = h.addVariable(lb=0, ub=highspy.kHighsInf,
8                   type=highspy.HighsVarType.kInteger, name="x1")
9 x2 = h.addVariable(lb=0, ub=highspy.kHighsInf,
10                  type=highspy.HighsVarType.kInteger, name="x2")
11
12 # Constraints
13 h.addConstr(2*x1 + 3*x2 <= 30) # (1) 2*x1 + 3*x2 <= 30
14 h.addConstr(3*x1 + 2*x2 <= 30) # (2) 3*x1 + 2*x2 <= 30
15 h.addConstr( x1 - x2 <= 5.5) # (3) x1 - x2 <= 5.5
16
17 # Define obj with priority (HiGHS minimises by default)
18 # Priority 1: Maximise z1 = x1 + x2
19 obj1 = highspy.HighsLinearObjective()
20 obj1.weight = 1
21 obj1.priority = 2 # Higher priority
22 obj1.abs_tolerance = 0
23 obj1.rel_tolerance = 0
24 obj1.coefficients = [-1.0, -1.0] # -x1-x2 (to max x1+x2)
25 obj1.offset = 0
26
27 # Priority 2: Minimise z2 = x1 + 3*x2
28 obj2 = highspy.HighsLinearObjective()
29 obj2.weight = 1
30 obj2.priority = 1 # Lower priority
31 obj2.abs_tolerance = 0
32 obj2.rel_tolerance = 0
33 obj2.coefficients = [1.0, 3.0] # x1+3*x2
34 obj2.offset = 0
35
36 # Add objectives to the model
37 h.addLinearObjective(obj1)
38 h.addLinearObjective(obj2)
39
40 # Solve: hierarchical optimisation (weighted disabled)
41 h.setOptionValue("blend_multi_objectives", False)
42 h.run()

```

Listing 15.3: Formulation with HiGHS and Python using highspy for hierarchical optimization.

This Python code given by Listing 15.3 illustrates hierarchical optimisation (objective 1 first, objective 2 second) using “highspy”, a Python SOL. By setting `blendmulti_objectives = False` (line 41), HiGHS performs hierarchical optimisation based on priority values assigned to each `HighsLinearObjective` (lines 17~34). The solver first optimizes the objective `z1` (maximization), then optimizes the objective `z2` (minimization) among all solutions that preserve the optimality of `z1`. Since all tolerance values are set to zero, no degradation of higher-priority objectives is allowed, and a strict hierarchical optimisation is applied.

15.2.4 COPT [14]

*Cardinal Optimizer*⁵ (COPT) is an emerging commercial optimisation platform developed by Cardinal Operations, with support for GPU-acceleration and COPT Studio, an integrated development environment for model development and debugging. Cardinal Optimizer (COPT) is particularly attractive for users in Asia.

For multi-objective optimisation, COPT provides native functionality through the `setObjectiveN()` API, supporting both hierarchical and weighted-sum methods. In hierarchical mode, COPT optimizes objectives sequentially based on priority values, offering fine-grained control through `MultiObjAbsTol` and `MultiObjRelTol` parameters that specify allowable degradation of higher-priority objectives when optimizing lower-priority ones. COPT does not provide built-in functionality for the computation of $\mathcal{Z}_N$.

```

1 import coptpy as cp
2 from coptpy import COPT
3
4 # Create COPT environment
5 env = cp.Envr()
6
7 # Create COPT model
8 model = env.createModel("BiObjectiveHierarchical")
9
10 # Decision variables (non-negative integers)
11 x1 = model.addVar(lb=0, vtype=COPT.INTEGER, name="x1")
12 x2 = model.addVar(lb=0, vtype=COPT.INTEGER, name="x2")
13
14 # Add constraints
15 model.addConstr(2*x1 + 3*x2 <= 30, name="constr1") # (1)
16 model.addConstr(3*x1 + 2*x2 <= 30, name="constr2") # (2)
17 model.addConstr(x1 - x2 <= 5.5, name="constr3") # (3)
18
19 # Define objective functions with priorities
20 # Objective 1: maximize z1 = x1 + x2 (high priority = 2)
21 model.setObjectiveN(
22     idx=0,
```

⁵ <https://www.copt.de/>

```

23     expr=x1 + x2,
24     sense=COPT.MAXIMIZE,
25     priority=2,
26     weight=1.0,
27     abstol=1e-6,
28     reltol=0.0 )
29
30 # Objective 2: minimize z2 = x1 + 3*x2 (low priority = 1)
31 model.setObjectiveN(
32     idx=1,
33     expr=x1 + 3*x2,
34     sense=COPT.MINIMIZE,
35     priority=1,
36     weight=1.0,
37     abstol=1e-6,
38     reltol=0.0 )
39
40 # Solve the model
41 model.solve()

```

Listing 15.4: Formulation with COPT and Python using `coptpy` for hierarchical optimization.

The Python code given by Listing 15.4 solves the illustrative example using the COPT solver with a hierarchical approach. The code utilizes COPT's multi-objective optimisation functionality through the `setObjectiveN` method (lines 21~38), which allows defining multiple objectives with different priorities. The first objective ($z1 = x1 + x2$) is set to be maximized with a higher priority level of 2, while the second objective ($z2 = x1 + 3*x2$) is set to be minimized with a lower priority level of 1. Each objective is configured with specific parameters including priority weights, absolute tolerance (`abstol`), and relative tolerance (`reltol`) to control the optimisation process. The hierarchical optimisation is executed through a single `solve()` call, which handles the hierarchical objective optimisation automatically based on the specified priorities.

15.2.5 Hexaly [34]

*Hexaly*⁶ is a modern commercial optimisation software suite, evolved from its predecessor *LocalSolver*. It is composed of *Hexaly Optimizer*, a global optimisation solver, and *Hexaly Modeler*, a modeling tool. The solver is built on a hybrid architecture blending exact algorithms (simplex, branch-and-bound, constraint propagation) with heuristics (local search, large neighborhood search). The modeler is accessible through multiple programming interfaces including *HXM*, the native Hexaly's declarative modeling language.

⁶ <https://www.hexaly.com/>

For multi-objective optimisation, Hexaly provides a hierarchical method where objectives are optimized in their order of declaration, see Listing 15.5. However, Hexaly does not provide built-in support for the computation of $\mathcal{Z}_N$.

```

1 function model() {
2   // Decision variables (non-negative integers)
3   x1 <- int(0, 100);
4   x2 <- int(0, 100);
5
6   // Constraints
7   constraint 2*x1 + 3*x2 <= 30;
8   constraint 3*x1 + 2*x2 <= 30;
9   constraint x1 - x2 <= 5.5;
10
11  // Objectives
12  z1 <- x1 + x2;
13  z2 <- x1 + 3*x2;
14
15  // Hierarchical optimization
16  maximize z1;      // First priority
17  minimize z2;      // Second priority
18 }

```

Listing 15.5: Formulation with Hexaly using HXM for hierarchical optimization.

In Listing 15.5, the function `model()` defines the optimisation problem using the “HXM” syntax. Decision variables `x1` and `x2` (lines 3~4) are declared as bounded integer variables in the interval $[0, 100]$. The three linear constraints are expressed using the `constraint` keyword (lines 7~9), and the two objectives are defined lines 12~13, represented by `z1` and `z2`. The hierarchical optimisation is implicitly specified through sequential objective declarations: `maximize z1` for the first objective, followed by `minimize z2` for the second objective. Hexaly automatically handles the hierarchical ordering, optimizing objectives in their declaration order, meaning it first finds the optimal value for `z1`, then optimizes `z2` while maintaining `z1` at its strict optimal level (no tolerance allowed). This approach avoids the need for artificial weighting schemes and produces a single solution that represents a trade-off according to the specified priority hierarchy.

Although Hexaly provides a high-level modeling language with HXM, its tight integration with the native solver and the absence of standard export mechanisms to MPS and LP formats limit its portability, placing it in the class of Solver-Oriented Interfaces.

15.2.6 FICO Xpress Optimiser [23]

*FICO Xpress Optimizer*⁷ is a comprehensive commercial optimisation suite, accessible through multiple modeling tools including *MOSEL*, the Xpress's native algebraic modeling language.

FICO Xpress Optimizer provides multi-objective optimisation capabilities through its hierarchical and blended objective approaches. Users can define multiple objective functions and optimize them either hierarchically, or simultaneously by combining them into a weighted sum. Additionally, Xpress MOSEL modeling language offers flexible syntax for declaring and manipulating multiple objectives. However, FICO Xpress Optimizer does not provide built-in functionality for the computation of $\mathcal{L}_N$.

```

1 model BiObjectiveLexicographic
2 uses "moxprs"
3
4 declarations
5     ! Decision variables
6     x1, x2: mpvar
7 end-declarations
8
9 ! Variables are non-negative integers
10 x1 is_integer
11 x2 is_integer
12 x1 >= 0
13 x2 >= 0
14
15 declarations
16     ! Objective functions
17     z1, z2: linctr
18 end-declarations
19
20 ! Define objective functions
21 z1 := x1 + x2
22 z2 := x1 + 3*x2
23
24 ! Define constraints
25 2*x1 + 3*x2 <= 30
26 3*x1 + 2*x2 <= 30
27 x1 - x2 <= 5.5
28
29 ! Hierarchical optimization
30 ! Priority 2 (higher): maximize z1
31 ! Priority 1 (lower): minimize z2
32 maximize([z1, z2], [objconfig(2, 1), objconfig(1, -1)])
33 end-model

```

Listing 15.6: Formulation with Xpress using MOSEL for hierarchical optimization.

⁷ <https://www.fico.com/en/products/fico-xpress-optimisation>

This MOSEL formulation for FICO Xpress in Listing 15.6 implements hierarchical optimisation using the `objconfig` function to specify objective priorities and directions. Decision variables `x1` and `x2` are declared as non-negative integer variables using the `mpvar` type with explicit bounds and integer constraints (lines 4~13). In lines 15~22, the two objective functions are defined as linear constraint objects (`linctr`). The `objconfig` function takes priority as its first argument (higher integers are optimized first) and weight as its second argument (positive for maximization, negative for minimization). Thus, `objconfig(2,1)` assigns priority 2 with positive weight to maximize `z1` first, while `objconfig(1,-1)` assigns priority 1 with negative weight to minimize `z2` second. The `maximize([z1,z2],[objconfig(2,1),objconfig(1,-1)])` statement (line 32) executes the hierarchical optimisation, ensuring that `z2` is optimized only after `z1` has reached its optimal value.

15.2.7 CPLEX Optimisation Studio [36]

*IBM ILOG CPLEX Optimisation Studio*⁸ is a leading commercial optimisation software suite that combines high-performance mathematical programming solvers with advanced modeling environments and APIs. It provides *CPLEX Optimizer* for a wide range of mathematical optimisation problems, *CP Optimizer* for constraint programming and constraint-based scheduling problems. It provides also multiple modeling interfaces including *OPL* (Optimisation Programming Language), CPLEX's native declarative modeling language integrated within CPLEX Studio IDE, offering mathematical syntax close to standard notation. CPLEX is compatible with third-party modeling languages including AMPL, GAMS, JuMP, and Pyomo through solver interfaces. This comprehensive ecosystem makes CPLEX one of the most widely adopted optimisation platforms in both academic research and industrial applications.

OPL (Optimisation Programming Language) is the AML integrated within CPLEX Optimisation Studio, and uses CPLEX Optimizer. For multi-objective optimisation, OPL supports weighted sum approaches for multiple comparable objectives, allowing users to blend objectives into a single optimisation function. Listing 15.7 implements a weighted sum approach for the illustrative example. OPL also integrates a particular scalarizing function called `staticLexFull` that allows users to define a multi-objective policy combining their goals, weights, priorities, and tolerances. This function allows, for example, hierarchical optimisation to be carried out where objectives are optimized hierarchically. Like many other commercial solvers, CPLEX does not provide built-in functionality for the computation of $\mathcal{Z}_N$.

⁸ <https://www.ibm.com/fr-fr/products/ilog-cplex-optimisation-studio>

```

1  /*****
2  * OPL 22.1.1.0 Model
3  *****/
4
5  // Decision variables
6      dvar int+ x1;
7      dvar int+ x2;
8
9  // Weighting of the sum
10     float weight1 = 0.6;
11     float weight2 = 0.4;
12
13 // Definition of objective functions
14     dexpr float z1 = x1 + x2;
15     dexpr float z2 = -1*x1 - 3*x2;
16
17 // Setting the problem to solve
18     maximize weight1*z1 + weight2*z2;
19     subject to {
20         2*x1 + 3*x2 <= 30;
21         3*x1 + 2*x2 <= 30;
22         x1 - x2 <= 5.5;
23     }

```

Listing 15.7: Formulation with CPLEX using OPL for the weighted sum optimization.

The OPL formulation given by Listing 15.7 shows the model implemented for the illustrative example. Two decision variables, x_1 and x_2 , in the form of non-negative integers (lines 2~3) and two objectives to maximize, z_1 and z_2 , are defined (lines 8~9). By assigning weights of 0.6 and 0.4 respectively (lines 5~6), the formulation scalarizes the multi-objective problem into a single objective function (line 11).

It should be noted that OPL implicitly invokes CPLEX as its underlying MILP solver, as OPL and CPLEX are parts of the IBM ILOG CPLEX Optimisation Studio and relies entirely on CPLEX's optimisation engine to solve linear, integer, and mixed-integer programming problems (LP, MILP, QP, MIQP). Therefore OPL does not satisfy the portability criterion associated with AMLs, as models formulated in OPL cannot be directly transferred to other solvers except through the export of standardized formats such as MPS.

15.2.8 AMPL [3]

*AMPL MP*⁹ (A Mathematical Programming Language) is a comprehensive commercial AML renowned for its expressive syntax that closely mirrors mathematical notation and its ability to interface with dozens of commercial and open-source solvers.

⁹ <https://ampl.com/>

For multi-objective optimisation, AMPL enables formulating hierarchical or blended multi-objective problems. For this, it provides native support through the suffix mechanism. By default, all objectives are blended together. Suffix `objweight` can be used to change the individual weights and objective senses, `objpriority` to specify hierarchical optimisation priorities, and `objabstol`, `objreلتol` to control allowable objective degradation. Advanced features include objective-specific solver options (time limits, MIP gaps) that can be set independently for each objective phase, particularly useful for controlling computational resources. But here again, AMPL does not provide built-in functionality for the computation of $\mathcal{Z}_N$.

```

1 # Variables
2 var x1 integer >= 0;
3 var x2 integer >= 0;
4
5 # Objectives
6 var z1 = x1 + x2;
7 var z2 = x1 + 3*x2;
8
9 # Constraints
10 subject to c1: 2*x1 + 3*x2 <= 30;
11 subject to c2: 3*x1 + 2*x2 <= 30;
12 subject to c3: x1 - x2 <= 5.5;
13
14 # to assign priority
15 suffix objpriority IN;
16
17 # most prioritary objective
18 maximize obj1: z1
19     suffix objpriority 10;
20
21 # least prioritary objective
22 minimize obj2: z2
23     suffix objpriority 1;
24
25 # Solver
26 option solver gurobi;
27
28 # Solver options: multi-objective and verbose output
29 option gurobi_options "obj:multi=2 outlev=1";
30
31 # Solve the multi-objective problem
32 solve;

```

Listing 15.8: Formulation with AMPL for the hierarchical optimization using Gurobi as solver.

The AMPL code given by Listing 15.8 models the illustrative example for the hierarchical optimisation. Multi-objective behavior is specified through AMPL suffixes attached to the objectives. In particular, the `objpriority` suffix (lines 15~23) assigns priorities to the objectives, indicating that `z1` must be optimized

first (higher priority), and that z_2 is optimized only among solutions that are optimal with respect to z_1 . In line 29, the parameter `obj:multi=2` informs the solver that a multi-objective problem is being provided (the value 2 corresponds to emulated multi-objective mode, the value 1 to native multi-objective mode if available, and the value 0 (default) does not activate multi-objective mode). Here, Gurobi is selected (line 26) as MIP solver for computing the corresponding hierarchical optimal solution (line 32).

15.2.9 AIMMS [2]

*AIMMS*¹⁰ (Advanced Integrated Multidimensional Modeling System) is a AML and integrated development environment for mathematical optimisation. It supports a wide range of optimisation problem, with interfaces to many solvers.

For multi-objective optimisation, AIMMS offers specialized functionality through its GMP (Generated Mathematical Program) library, enabling both blended and hierarchical objective handling. Currently, multi-objective optimisation in AIMMS is supported exclusively for linear models solved only with CPLEX or Gurobi. Also, AIMMS does not provide built-in functionality for the computation of $\mathcal{Z}_N$.

Finally, it was not possible to analyze this software using the illustrative example because AIMMS is not available on macOS and Linux operating systems.

15.2.10 GAMS [24]

*GAMS*¹¹ (General Algebraic Modeling System) is one of the oldest and most known AMLs. It supports linear, mixed-integer, and nonlinear problems, making it suitable for a broad range of applications.

GAMS allows the explicit declaration of multiple objective equations, which are handled natively and passed directly to the multi-objective internal library, “moo”, which contains procedures such as `EpsConstraint` which implement an ε -constraint method. This approach generates the set of nondominated points for a bi-objective optimisation problem by solving a series of single-objective problems. The syntax for calling the moo library with `EpsConstraint` method is the following: `$libInclude moo method model modeltype idxObj dirObj varObj nbMaxPoints nondominatedSet options`

Its tight integration with many MILP solvers allows users to exploit solver-level multi-objective capabilities transparently.

¹⁰ <https://www.aimms.com/>

¹¹ <https://www.gams.com/>

```

1 Set
2   k /k1*k2/
3   p /point1*point100/;
4
5 Parameter
6   dir(k)
7   Zn(p,k);
8
9   dir['k1'] = 1;
10  dir['k2'] = -1;
11
12 Integer Variable
13   x1, x2;
14
15 Free Variable
16   z(k);
17
18 Equation
19   obj1, obj2
20   cst1, cst2, cst3;
21
22   obj1.. z('k1') =e= x1 + x2;
23   obj2.. z('k2') =e= x1 + 3*x2;
24
25   cst1.. 2*x1 + 3*x2 =l= 30;
26   cst2.. 3*x1 + 2*x2 =l= 30;
27   cst3.. x1 - x2 =l= 5.5;
28
29 model
30   MOMP /all/;
31
32 * Call the moo library with EpsConstraint method
33 $libInclude moo EpsConstraint MOMP MIP k dir z p Zn -
34   gridpoints=30 -solver=cplex
35
36 display Zn;

```

Listing 15.9: Formulation with GAMS using the EpsConstraint method from the “moo” library, and CPLEX as solver.

This GAMS formulation reported in Listing 15.9 implements the bi-objective illustrative example. The indices k , with $k1 \leq k \leq k2$, and p with $point1 \leq p \leq point100$, are defined (lines 1~3). The parameter `dir` defines the direction of optimisation for each objective, with 1 to maximize and -1 to minimize (lines 6, 9, 10). The parameter `Zn` in line 7 is a finite two-dimensional structure for storing (at most 100) non-dominated points. Two non-negative integer decision variables ($x1, x2$) (lines 12~13) are then declared. The two objectives are specified by two objective free variables $z(k)$ (lines 15–16) and the two expressions (lines 19, 22, 23). The feasible region $\mathcal{X}$ is set by three constraints (lines 20, 25–27).

The GAMS `moo` library provides a built-in function `EpsConstraint` that implements the ε -constraint method, which generates the nondominated set by itera-

tively optimizing one objective while constraining the other. The results are returned by the data structure named here Y_n , provided as a parameter. The `$libInclude` command (line 33) invokes CPLEX as the underlying MIP solver to compute up to 30 nondominated points, providing to the decision-maker a view of the trade-offs between the two competing objectives.

GAMS is the only commercial software that allows the computation of the set of nondominated points.

15.2.11 Pyomo [33], PuLP [49], CVXPY [16], and MOCVXPY [50]

*Pyomo*¹², *PuLP*¹³, *CVXPY*¹⁴, and *MOCVXPY*¹⁵ are Python-based open-source software packages that support a diverse set of optimisation capabilities for formulating, and solving mathematical optimisation models. *Pyomo*, *PuLP*, and *CVXPY* are among the most well-established Python packages for optimisation, while *MOCVXPY* has been very recently introduced and is specifically designed for dealing with multiple objectives.

Multi-objective optimisation capabilities are non-existent for PuLP, very limited for Pyomo. Indeed, the current version of PuLP (3.3.0) provides no native support, whereas the current version of Pyomo (6.9.5) allows multiple objectives to be defined but only one can be active at a time. As illustrated in Listing 15.10, if both `model.obj1` and `model.obj2` have been declared using `Objective` method (lines 3~4), the user must manually deactivate objectives, except one e.g. `model.obj2` (lines 5~6), to ensure that only one objective is passed to the solver as shown in this snippet. As no native multi-objective algorithm is available in Pyomo, the user has to implement all operations for multi-objective optimisation (such as a hierarchical method, a lexicographic method, an epsilon-constraint method, a weighted sum method, or a more advanced method) each of which may be found as an individual contribution, as *PyAUGMECON*¹⁶, the augmented ϵ -constraint (AUGMECON) solver for Pyomo.

```

1 import pyomo.environ as pyo
2 model = pyo.ConcreteModel()
3 model.obj1 = pyo.Objective(expr = 0)
4 model.obj2 = pyo.Objective(expr = 0)
5 model.obj1.deactivate()
6 model.obj2.activate()

```

Listing 15.10: Declaration of functions and desactivation of all but one with Pyomo.

¹² <https://www.pyomo.org>

¹³ <https://pypi.org/project/PuLP/>

¹⁴ <https://www.cvxpy.org/>

¹⁵ <https://github.com/salomon1/mocvxdpy>

¹⁶ <https://github.com/wouterbles/pyaugmecon>

Designed for convex optimisation, the current version of CVXPY (1.7.5) provides native support for multi-objective convex problems by allowing to define lists of objectives, and solving it through a weighted sum method with the function `scalarize.weighted_sum`.

```

1 import cvxpy as cp
2 from cvxpy.transforms import scalarize
3
4 # Definition of variables (non-negative integers)
5 x1 = cp.Variable(integer=True, nonneg=True)
6 x2 = cp.Variable(integer=True, nonneg=True)
7
8 # Definition of objectives
9 z1 = x1 + x2 # to maximise
10 z2 = x1 + 3*x2 # to minimise
11
12 # Definition of constraints
13 constraints = [
14     2*x1 + 3*x2 <= 30, # (1)
15     3*x1 + 2*x2 <= 30, # (2)
16     x1 - x2 <= 5.5, # (3)
17 ]
18
19 # Weight for each objective
20 lambda1 = 0.75 # weight for z1
21 lambda2 = 0.25 # weight for z2
22
23 # Creation of the objective function with 'scalarize'
24 objectives = [z1, -z2]
25 weights = [lambda1, lambda2]
26
27 problem = cp.Problem(
28     cp.Maximize(scalarize.weighted_sum(objectives, weights)),
29     constraints
30 )
31 problem.solve(solver=cp.HIGHS)

```

Listing 15.11: CVXPY multi-objective formulation and HiGHS used as solver

This Python code using CVXPY implements a weighted sum scalarization. The objectives are defined as linear expressions ($z_1 = x_1 + x_2$ and $z_2 = x_1 + 3x_2$) and combined using the `scalarize.weighted_sum()` function with weights $\lambda_1 = 0.75$ and $\lambda_2 = 0.25$. Since CVXPY only supports single-objective optimisation, the two objectives are converted into a scalar objective by maximizing the weighted combination.

This section ends with MOCVXPY, an extension for CVXPY introduced in 2025 that enables the formulation and solution of multi-objective convex optimisation problems. The library currently supports three algorithms: the Multiobjective Optimisation by Norm Min Optimisation (MONMO) algorithm, the Multiobjective Optimisation by Vertex Selection (MOVS) algorithm, and AdEnA [42] which is an enclosure-based framework that refines an enclosure of the non-dominated set

along the iterations. Unlike other software, *movxpy* is not yet available for installation from PyPI platform as of January 2026; its test will not be presented in this chapter.

15.2.12 BENSOLVE [42], Inner [15], and PaMILO [10]

*BENSOLVE*¹⁷, *Inner*¹⁸, and *PaMILO*¹⁹ are three MOMP solvers that share the ability to compute extreme supported non-dominated vertices for continuous linear problems (all three) and for discrete/mixed integer linear problems in the case of PaMILO. All three are based on ideas originating from the work of Harold P. Benson [8].

BENSOLVE is specialized for vector linear programs (VLP) and multiple objective linear programs (MOLP) based on Benson’s algorithm and its extensions. It is developed by the Friedrich Schiller University Jena’s mathematical optimisation research group (Germany). Users interact with the solver through command-line options. The typical workflow involves creating a VLP-format input file, running BENSOLVE with appropriate options, and analyzing the output files.

The Inner solver implements an inner approximation algorithm for solving linear multi-objective optimisation problems (MOLP). It is developed by the HUN-REN Alfréd Rényi Institute of Mathematics, Budapest (Hungary). Compared to BENSOLVE, which implements Benson’s outer approximation algorithm, INNER uses the conceptually opposite approach of inner approximation, building the solution polytope from inside rather than from outside.

These first two solvers are under the GNU GPL license, coded in C (note that BENSOLVE was implemented in Matlab before version 2.0.0), and utilize GLPK (GNU Linear Programming Kit) for linear programming operations. As a prerequisite for using BENSOLVE or Inner, GLPK must be installed. Both solvers rely on the same VLP input formats (see Listing 15.12 corresponding to the bi-objective illustrative example). A problem instance in VLP format is stored as a plain ASCII text file containing lines of several types. Note that the VLP input format, while powerful, is technically complex and not particularly human-friendly.

```

1 c PROBLEM LINE: class, direction, rows, cols, nz, obj,
   objnz
2 p vlp max 3 2 6 2 4
3
4 c OBJECTIVE COEFFICIENTS
5 c Format: o objective_index variable_index coefficient
6 c Objective 1: max z1 = x1 + x2
7 o 1 1 1
8 o 1 2 1

```

¹⁷ <https://users.fmi.uni-jena.de/~sa67leb/bensolve/>

¹⁸ <https://github.com/lcsirmaz/inner>

¹⁹ <https://github.com/FritzBo/PaMILO>

```

9
10 c Objective 2: min z2 = x1 + 3*x2 => change to max(-z2)
11 o 2 1 -1
12 o 2 2 -3
13
14 c CONSTRAINT COEFFICIENTS
15 c Format: a row_index variable_index coefficient
16
17 c Constraint 1: 2*x1 + 3*x2 <= 30
18 a 1 1 2
19 a 1 2 3
20
21 c Constraint 2: 3*x1 + 2*x2 <= 30
22 a 2 1 3
23 a 2 2 2
24
25 c Constraint 3: x1 - x2 <= 5.5
26 a 3 1 1
27 a 3 2 -1
28
29 c ROW DESCRIPTORS (constraints)
30 c Format: i row_index type [value1 [value2]]
31 c Types: f=free, l=lower, u=upper, d=double, s=equal
32
33 c Constraint 1: -inf to 30
34 i 1 u 30
35
36 c Constraint 2: -inf to 30
37 i 2 u 30
38
39 c Constraint 3: -inf to 5.5
40 i 3 u 5.5
41
42 c COLUMN DESCRIPTORS (variables)
43 c Format: j column_index type [value1 [value2]]
44 c Default: j COL 1 0 (lower bound 0)
45
46 c Variable x1 >= 0
47 j 1 l 0
48
49 c Variable x2 >= 0
50 j 2 l 0
51
52 c END OF FILE
53 e

```

Listing 15.12: VLP input format of BENSOLVE and Inner.

Here again, the illustrative problem cannot be solved with BENSOLVE or Inner because they do not natively support integer decision variables. Thus, only the continuous relaxation of the original integer problem can be addressed, which is given by Listing 15.12 with the VLP input format required by solvers. Each symbol in the first column of the VLP format has a particular role:

- c *comment line*: used to document the file without affecting the problem definition.
- p *problem line*: defines the global characteristics: type (vlp), direction (max/min), number of rows, columns, non-zero elements, objectives, and non-zero objective coefficients.
- o *objective coefficient*: specifies a coefficient in an objective function with the format: objective_index variable_index coefficient.
- a *constraint coefficient*: defines a coefficient in a constraint with the format: row_index variable_index coefficient.
- i *row descriptor*: specifies the type and bounds of a constraint (f=free, l=lower bound, u=upper bound, d=double bounds, s=equality).
- j *column descriptor*: defines the type and bounds of a variable with the same types as rows.
- e *end of file*: marks the end of the problem definition.

The third is PaMILO (Parametric Mixed Integer Linear Optimisation), a solver designed for multi-objective mixed integer linear programs (MOMILPs) and multi-objective mixed integer quadratic programs (MOMIQPs) based on the dual Benson algorithm and its extensions. It is developed by the University of Osnabrück's theoretical computer science group (Germany). PaMILO works with Gurobi (preferably) or CPLEX for underlying optimisation, which must be installed beforehand. Users interact with the solver also through command-line options and must provide instances in a format supported such as LP extended to multiple objectives. The typical workflow involves preparing an instance file, running PaMILO with appropriate parameters, and analyzing the output files. The programming language of PaMILO is C++, and the solver can be used subject to an academic variant of the MIT software license.

```

1 Maximize multi-objectives
2   obj1: Priority=0 Weight=1.0 AbsTol=0 RelTol=0
3     + 1 x1 + 1 x2
4   obj2: Priority=0 Weight=1.0 AbsTol=0 RelTol=0
5     - 1 x1 - 3 x2
6 Subject to
7   c1: 2 x1 + 3 x2 <= 30
8   c2: 3 x1 + 2 x2 <= 30
9   c3: 1 x1 - 1 x2 <= 5.5
10 Bounds
11   x1 >= 0
12   x2 >= 0
13 General
14   x1
15   x2
16 End

```

Listing 15.13: lp input format with multiple objectives of Gurobi used by PaMILO.

This bi-objective LP formulation given by Listing 15.13 defines two objectives x_1+x_2 and $-x_1-3x_2$ to maximize (lines 3~7), subject to three linear con-

straints (lines 8~11) on the non-negative integer decision variables x_1 and x_2 (lines 12~17).

These three software can be used either (1) as a standalone solver, or (2) as a subroutine within a more general algorithm, for example to compute the convex relaxation of a multi-objective problem with integer variables as in [4] for BENSOLVE, or (3) as the computational core of *BENSOLVE TOOLS*, an Octave/Matlab package that extends its functionality toward polyhedral calculus and global optimisation capabilities.

15.2.13 SCIP [1] with PolySCIP [11]

*SCIP*²⁰ (Solving Constraint Integer Programs) is an open-source optimisation framework released under the Apache 2.0 license, and developed primarily at the Zuse Institute Berlin (Germany). It is capable of solving various optimisation problems through sophisticated algorithms with extensive plugin architecture. It is part of the *SCIP Optimisation Suite*, a comprehensive toolbox that includes the mathematical programming language *ZIMPL*²¹.

For multi-objective optimisation, SCIP provides *PolySCIP*²², a specialized solver for multi-objective integer and linear programming that aims at computing the set of nondominated points $\mathcal{Z}_N$. PolySCIP can compute extreme supported nondominated vertices for continuous linear problems with an arbitrary number of objectives, and supported plus non-supported non-dominated points for bi-objective and tri-objective integer programs. PolySCIP's file format is based on the MOP format which is an extension of the MPS format, and supports problem classes including multi-objective linear programs (MOLP), binary programs (MOBP), integer programs (MOIP), and mixed-integer programs (MOMIP).

The recommended workflow to model a multiobjective optimisation problem with p objectives and n variables, and to solve it is the following:

1. Write the model with *ZIMLP* with all objectives in the same direction (see Listing 15.14);
save this file as e.g. `moip.zlp`
2. Generate the `.mop` file with the Python tool coming with PolySCIP:
`python3 mult_zimpl_to_mop.py moip.zlp --path_to_zimpl bin/`
3. Solve the multi-objective mathematical problem with *PolySCIP*:
`polyscip moip.mop`

PolySCIP will output the set of nondominated points for this MOMP, i.e. objective values $(z_1, \dots, z_p)$ and variable values $(x_1, \dots, x_n)$ of the corresponding efficient solutions.

²⁰ <https://www.scipopt.org/>

²¹ <https://zimpl.zib.de/>

²² <https://polyscip.zib.de/>

PolySCIP represents one of the first open-source tools capable of computing $\mathcal{Z}_N$ for multi-objective mixed-integer programs. However, the development of PolySCIP (currently the version 2.0 is available online) has been inactive since 2017, and its installation with recent versions of SCIP is currently not smooth and may even be challenging.

```

1 # DECISION VARIABLES (non-negative integers)
2 var x1 integer >= 0;
3 var x2 integer >= 0;
4
5 # OBJECTIVE FUNCTIONS
6 minimize Obj1: -x1 - x2;
7 Obj2: +x1 + 3*x2;
8
9 # CONSTRAINTS
10 subto c1: 2*x1 + 3*x2 <= 30;
11 subto c2: 3*x1 + 2*x2 <= 30;
12 subto c3:  x1 -   x2 <= 5.5;

```

Listing 15.14: Formulation with ZIMPL for computing $\mathcal{Z}_N$ using PolySCIP as solver.

The Listing 15.14 shows the illustrative problem expressed with the ZIMLP modeling language. The two objective functions are defined in accordance with ZIMLP's minimization-oriented syntax (lines 5~8). Both objectives are declared sequentially under the `minimize` keyword (lines 20~22). The constraints are defined in standard ZIMLP formalism. The MOP file generated by the Python tool, corresponding to the ZIMLP code, is the following:

```

1 NAME          BIOBJECTIVE
2 OBJSENSE
3 MIN
4 ROWS
5   N   Obj1
6   N   Obj2
7   L   c1
8   L   c2
9   L   c3
10 COLUMNS
11     x#1      Obj1      -1
12     x#1      Obj2       1
13     x#1      c1        2
14     x#1      c2        3
15     x#1      c3        1
16     x#2      Obj1      -1
17     x#2      Obj2       3
18     x#2      c1        3
19     x#2      c2        2
20     x#2      c3       -1
21 RHS
22     RHS      c1        30
23     RHS      c2        30

```

```

24      RHS      c3      5.5
25 BOUNDS
26   LI BOUND    x#1      0
27   LI BOUND    x#2      0
28 ENDATA

```

Listing 15.15: MOP file for PolySCIP

In the ROWS section, the tag N (for "No constraint" i.e. a free row) is used for the objective functions. Listing 15.16 shows the output reported by PolySCIP.

```

1 SCIP Status      : problem is solved [optimal solution
   found]
2 Solving Time (sec) : 0.00
3 Solving Nodes      : 0
4 Primal Bound       : +2.000000000000000e+00 (2 solutions)
5 Dual Bound        : +2.000000000000000e+00
6 Gap               : 0.00 %
7 PolySCIP Status: Successfully finished
8 Number of bounded results: 13
9 Number of unbounded results: 0
10 [ -12 24 ] x#1=6 x#2=6
11 [ 0 0 ]
12 [ -7 9 ] x#1=6 x#2=1
13 [ -9 13 ] x#1=7 x#2=2
14 [ -11 17 ] x#1=8 x#2=3
15 [ -10 16 ] x#1=7 x#2=3
16 [ -8 12 ] x#1=6 x#2=2
17 [ -4 4 ] x#1=4
18 [ -5 5 ] x#1=5
19 [ -6 8 ] x#1=5 x#2=1
20 [ -2 2 ] x#1=2
21 [ -3 3 ] x#1=3
22 [ -1 1 ] x#1=1

```

Listing 15.16: Output of PolySCIP

15.2.14 JuMP [43] with MultiObjectiveAlgorithms [18, 19]

*JuMP*²³ (Julia for Mathematical Programming) [20] is a free open-source Julia-based AML under the MPL-2.0 software license, offering an expressive algebraic syntax that interfaces with more than fifty commercial and open-source optimisation solvers across plenty of problem classes. JuMP originates from research conducted at the Massachusetts Institute of Technology (USA), and its first public version was released in 2013.

Developments around JuMP aimed at solving multi-objective optimisation can be broadly separated into two distinct periods.

²³ <https://jump.dev/JuMP.jl/stable/>

- Period 2015-2023.

The first period took place in the context of the ANR-DFG research project “vOpt” (2015–2019), during which the *vOptSolver*²⁴ ecosystem [25] was developed in Julia by the Nantes University’s multi-objective optimisation research group (France), whose main objective is to serve as a platform to facilitate the integration and evaluation of algorithms developed by the project. *vOptSolver* consists of two distinct Julia packages. The first one, *vOptGeneric*²⁵, is devoted to *multi-objective non-structured problems* and extends JuMP to support the modeling of a problem, its solving using a given algorithm, and the analysis of the resulting sets of points and solutions. It integrates classic multi-objective optimisation algorithms such as the lexicographic optimisation [37], the Aneja-Nair method [5], the ε -constraint method [32], the Chalmet method [12], among others. The second package, *vOptSpecific*²⁶, targets *multi-objective structured problems* (currently including multi-objective version of linear assignment, scheduling, knapsack, and facility location problems). It provides an API implementing state-of-the-art algorithms tailored to these classes of combinatorial problems. Both packages are registered in the Julia General Registry and were first publicly presented in 2017 [26, 27].

- Period 2019-today.

The second period emerged in the context of the major refactoring of JuMP carried out by the international community of active JuMP developers, marked by the transition from the `MathProgBase.jl` abstraction layer to `MathOptInterface.jl` (MOI)²⁷ [41], a new one designed to provide a unified interface to mathematical optimisation solvers. Discussions began in 2019 regarding the integration of native multi-objective optimisation features into JuMP. These efforts resulted in support for these features starting with JuMP version 1.8.0, through the use of vector-valued objective functions. In parallel, the development of the `MultiObjectiveAlgorithms.jl` (MOA)²⁸ package was initiated and publicly presented in 2023 [18]. MOA is licensed under the MPL 2.0 License and acts as a meta-solver by iteratively scalarizing multi-objective problems into sequences of single-objective subproblems that can be solved by any MIP solver compatible with JuMP. The package [19] includes algorithms already available in *vOptGeneric*, as well as several classic and cutting-edge algorithms designed for problems involving more than two objectives: Hierarchical, RandomWeighting [24], Sandwiching [40], DominguezRios [17], Kirlik-Sayin [38], Tamby-Vanderpooten [52]. Consequently *vOptGeneric.jl* is deprecated since 2023.

²⁴ <https://github.com/vOptSolver>

²⁵ <https://github.com/vOptSolver/vOptGeneric.jl>

²⁶ <https://github.com/vOptSolver/vOptSpecific.jl>

²⁷ <https://github.com/jump-dev/MathOptInterface.jl>

²⁸ <https://jump.dev/JuMP.jl/stable/packages/MultiObjectiveAlgorithms/>

`JuMP.jl` (**JuMP**), `MultiObjectiveAlgorithms.jl` (**MOA**) and wrappers to MIP solvers as `CPLEX.jl`, `Gurobi.jl`, `GLPK.jl` or again `HiGHS.jl` are registered in the General Julia package registry. Consequently using JuMP and MOA with a MIP solver during a working session is extremely simple. In the following steps, the instructions required to model, solve, and collect the efficient solutions and nondominated points for the illustrative example are provided step by step. Before these steps, a Julia distribution has to be properly installed on the user computer (see the section “Download” at <https://julialang.org/>) and a Julia working environment (a *REPL* command-line interface, a *Jupyter* notebook, the *VSCode* integrated development environment, etc.) has to be ready.

Step 0.

The required packages only need to be added to the Julia user environment once, see Listing 15.17. Moreover, if the MIP solver is open source, such as HiGHS or GLPK, the user does not need to install the solver separately beforehand.

```
1 # Adding packages to the Julia's user environment
2 using Pkg
3 Pkg.add("JuMP")
4 Pkg.add("MultiObjectiveAlgorithms")
5 Pkg.add("HiGHS")
```

Listing 15.17: Julia instructions using `Pkg.add` to install the required packages. These commands only need to be executed once to add the packages to the Julia environment.

Here, JuMP and MultiObjectiveAlgorithms are added, as well as HiGHS which is chosen here to be used as MIP solver.

Step 1.

First, the user must declare that these three packages (JuMP, MultiObjectiveAlgorithms, HiGHS) will be used. This is done once per session using the Julia instructions given in Listing 15.18.

```
1 # Using packages in a Julia's working environment
2 using JuMP
3 import MultiObjectiveAlgorithms as MOA
4 using HiGHS
```

Listing 15.18: Declaration of packages to use

Step 2.

Next, variables, objectives, and constraints of the multi-objective model are set up using the JuMP modeling language, which directly mirrors the mathematical formulation of the problem.

```

1  # Model to setup with JuMP
2  m = Model()
3
4  # Decision variables
5  @variable(m, x1>=0, Int)
6  @variable(m, x2>=0, Int)
7
8  # Vector of objective functions to maximize
9  @expression(m, z1, x1+ x2)
10 @expression(m, z2, x1+3x2)
11 @objective(m, Max, [z1,-z2])
12
13 # Constraints
14 @constraint(m, 2x1+3x2 <= 30)
15 @constraint(m, 3x1+2x2 <= 30)
16 @constraint(m, x1- x2 <= 5.5)

```

Listing 15.19: Formulation with JuMP

In Listing 15.19, line 2 declares an empty JuMP model named `m`. Lines 5~6 define two nonnegative integer decision variables `x1` and `x2`. The objectives are formulated as a vector (lines 9~11); the first component maximizes $z1=x1+x2$, while the second minimizes $z2=x1+3x2$, implemented by maximizing $[z1, -z2]$. Lines 14~16 express three linear constraints defining the feasible region $\mathcal{X}$.

Step 3.

At this stage, the user has to set up the MILP solver to be used and to set up the solution algorithm to be activated, plus possibly additional optional attributes supported by the algorithms (as a time limit), before finally running the optimisation process.

```

1  # select the MILP solver to invoke
2  set_optimizer(m, ()->MOA.Optimizer(HiGHS.Optimizer))
3  # select the algorithm to apply
4  set_attribute(m, MOA.Algorithm(), MOA.EpsilonConstraint())
5  # compute the results
6  optimize!(m)

```

Listing 15.20: Setup a resolution with JuMP and MOA

In line 2 of Listing 15.20, the HiGHS MILP solver is selected and wrapped by the MOA optimizer. Then, the epsilon-constraint algorithm is specified as the solution method to solve the problem (line 4), with the default step equal to 1. Finally, the `optimize!` (line 6) command launches the optimisation. One point to take

into consideration by the user when selecting the solver and the algorithm is to ensure that choices are compliant with the class of problems under consideration and consistent with its structure (e.g., convexity, integrality, or number of objectives).

Step 4.

Finally, the user has access to features for collecting optimisation results, in particular the number of elements comprising the results, the values of non-dominated points, and the values of efficient solutions.

```

1 using Printf
2 for i in 1:result_count(m)
3     z1Opt = Int( objective_value(m; result=i)[1] )
4     z2Opt = Int( -objective_value(m; result=i)[2] )
5     x1Opt = Int( value(x1; result=i) )
6     x2Opt = Int( value(x2; result=i) )
7     @printf("%2d: x=(%d,%d) z=(%2d,%2d)\n",
8             i, x1Opt, x2Opt, z1Opt, z2Opt)
9 end

```

Listing 15.21: Query the optimisation results with JuMP and MOA

The code illustrated by Listing 15.21 iterates through all results found using `result_count(m)` to get the total number of trade-offs. For each result indexed by `i`, it extracts the objective values using `objective_value(m; result=i)`, retrieving `z1` directly from index `[1]` and `z2` by negating index `[2]` (since the model maximized $-z2$ instead of minimizing $z2$). The corresponding decision variable values are obtained via `value(x1; result=i)` and `value(x2; result=i)`. All values are printed (see Listing 15.22), providing to the decision-maker the complete trade-off information, both in decision space and objective space. `@printf` is used to tabulate the displayed results, which is why the `Printf` package is invoked in line 1.

```

1 1: x=(6,6) z=(12,24)
2 2: x=(8,3) z=(11,17)
3 3: x=(7,3) z=(10,16)
4 4: x=(7,2) z=( 9,13)
5 5: x=(6,2) z=( 8,12)
6 6: x=(6,1) z=( 7, 9)
7 7: x=(5,1) z=( 6, 8)
8 8: x=(5,0) z=( 5, 5)
9 9: x=(4,0) z=( 4, 4)
10 10: x=(3,0) z=( 3, 3)
11 11: x=(2,0) z=( 2, 2)
12 12: x=(1,0) z=( 1, 1)
13 13: x=(0,0) z=( 0, 0)

```

Listing 15.22: Output of JuMP

15.3 Lessons learned

15.3.1 Summary

We would like to remind you that the review focused on solver limited to deterministic optimizers, and software dealing with classes of linear problems with continuous, discrete, or mixed variables, involving two or more objectives.

<i>Software</i>	<i>Commercial</i>	<i>Open-source</i>	<i>Modeler</i>	<i>Solver</i>	<i># Trade-offs</i>
MATLAB	✓	–	SOI	–	≥ 1
Gurobi Optimizer	✓	–	SOI	generic	1
HiGHS	–	✓	SOI	generic	1
COPT	✓	–	SOI	generic	1
Hexaly	✓	–	SOI	generic	1
FICO Xpress Optimizer	✓	–	AML+SOI	generic	1
CPLEX Optimisation Studio	✓	–	AML+SOI	generic	1
AMPL	✓	–	AML	–	1
AIMMS	✓	–	AML	–	1
GAMS	✓	–	AML	–	≥ 1
Pyomo	–	✓	AML	–	–
PuLP	–	✓	AML	–	–
CVXPY	–	✓	AML	specific	1
MOCVXPY	–	✓	AML	specific	≥ 1
BENSOLVE	–	✓	–	specific	≥ 1
INNER	–	✓	–	specific	≥ 1
PaMILO	–	✓	–	specific	≥ 1
SCIP with ZIMLP and PolySCIP	–	✓	AML	specific	≥ 1
JuMP with MultiObjectiveAlgorithms	–	✓	AML	meta-solver	≥ 1

Table 15.1: Summary of software examined.

Table 15.1 summarizes the software examined for modeling or solving mathematical programs that natively support multi-objective optimisation features.

15.3.2 Discussion

The examination of the different modeling tools and solvers considered in this survey leads to several observations. First, the multi-objective capabilities of software originally designed for single-objective optimisation remain largely rudimentary. Almost all of them are limited to basic approaches –such as hierarchical optimisation or weighted-sum methods– that compute a single solution (e.g., Gurobi, CPLEX, Hexaly, COPT, HiGHS, Xpress, AMPL, AIMMS, among others). Such

approaches are inherently insufficient to capture all, or even a representative subset, of the trade-off solutions that characterize multi-objective optimisation problems. A notable exception is GAMS, which, through an external library, provides an epsilon-constraint method capable of computing trade-offs for multi-objective problems (with MATLAB also offering related functionality, albeit limited to approximations).

Second, for several tools, multi-objective processing is delegated to external routines (as in the cases of MATLAB, or GAMS), which weakens the continuity of the modeling–solving workflow. Conversely, software specifically designed for multi-objective optimisation often suffers from limited or cumbersome modeling capabilities, reduced ease of use, reliance on technically complex and non-user-friendly exchange formats (such as VLP or MOP), or the need for intermediate steps that disrupt the direct interaction between modeler and solver (as observed with ZIMPL and PolySCIP).

The only clear exception to this set of limitations is the Julia-based software stack composed of `JuMP.jl` (JuMP) and `MultiObjectiveAlgorithms.jl` (MOA). Together, they form a modern and flexible algebraic modeling environment with native support for multi-objective modeling, a rich collection of classical and advanced algorithms capable of computing trade-offs for problems with two or more objectives, and seamless access to a wide range of MILP solvers, both commercial and open-source.

The JuMP and MultiObjectiveAlgorithms combination makes it possible to address (1) large-scale, complex industrial problems, whether the goal is to compute a single compromise solution or all trade-off solutions; (2) educational and training activities in multi-objective optimisation, without requiring learners to possess advanced technical expertise in computer science; and (3) the evaluation and comparison of new algorithms arising from scientific research, thanks to the open architecture of MultiObjectiveAlgorithms and the seamless access to these algorithms through the JuMP algebraic modeling language.

One may reasonably expect that, in the near future, existing multi-objective optimisation tools such as BENSOLVE, INNER, PolySCIP, or PaMILO—among others—will also be integrated into the JuMP/MultiObjectiveAlgorithms ecosystem. The added value would be immediate for both sides: for instance, BENSOLVE would benefit from access to a modern algebraic modeling language through JuMP, while MultiObjectiveAlgorithms would be enriched with advanced solution techniques not yet available, such as the many variants of Benson’s method, multi-objective simplex algorithms, and related state-of-the-art approaches.

15.4 Exercises

The bi-objective illustrative example given in Section 15.1.4 is an *explicit* formulation, i.e. a description in extension of the mathematical model composed of variable(s), function(s), constraint(s), and values.

A formulation is said to be *implicit* when the description of the mathematical model composed of variable(s), function(s), constraint(s), is compact and without value, these latter being externalized and represented in the formulation by parameters. The implicit formulation of the bi-objective illustrative example is thus:

$$\begin{aligned} \max z_k &= \sum_{j=1}^n c_j^k x_j & k &= 1, \dots, p \\ \text{s.t.} \quad & \sum_{j=1}^n t_{i,j} x_j \leq d_i & i &= 1, \dots, l \\ & x_j \in \mathbb{N} & j &= 1, \dots, n \end{aligned}$$

with all the c_j^k , $t_{i,j}$, d_i , n , l , and p set-up before.

Questions:

1. Using software of your choice other than JuMP and MOA, formulate the problem implicitly and solve it by computing a single trade-off; then answer the same question using JuMP with MOA and a generic MILP solver of your choice. Hint: Consult the documentation for the selected software for instructions on how to write an implicit formulation.
2. The same question as before but now for computing all trade-offs. Report the corresponding efficient solutions and nondominated points.
3. The same question as before but now using a different solver.

Acknowledgments

I express my sincere thanks to all those who contributed to the development of the packages that compose `vOptSolver`, in particular Gauthier Soleilhac who was in charge of the deployment of the packages. I am also grateful to the active community behind the JuMP ecosystem, especially Miles Lubin, Oscar Dowson, and Gökhan Kof, with whom I have had the opportunity to discuss multi-objective aspects of JuMP over several years. My thanks also go to the successive cohorts of second-year master's students in optimisation and operations research, with whom I have explored and developed topics related to multi-objective optimisation software over the years. My thanks also go to all those who gave me comments and feedback on the chapter. Finally, I thank Matthias Ehrgott for inviting me to write this chapter, as well as the whole team coordinating this book project.

References

- [1] Tobias Achterberg. “SCIP: solving constraint integer programs”. In: *Mathematical Programming Computation* 1.1 (July 2009), pp. 1–41. ISSN: 1867-2957. DOI: 10.1007/s12532-008-0001-1. URL: <https://doi.org/10.1007/s12532-008-0001-1>.
- [2] AIMMS. *Multiple objective*. Date visited: 2025-12-31. 2025. URL: <https://documentation.aimms.com/language-reference/optimization-modeling-components/implementing-advanced-algorithms-for-mathematical-programs/multi-objective-optimization.html>.
- [3] AMPL Optimization Inc. *Multiple objective*. Date visited: 2025-12-31. 2025. URL: <https://mp.ampl.com/modeling-mo.html>.
- [4] Duleabom An et al. “A matheuristic for tri-objective binary integer linear programming”. In: *Computers & Operations Research* 161 (2024), p. 106397. DOI: 10.1016/J.COR.2023.106397. URL: <https://doi.org/10.1016/j.cor.2023.106397>.
- [5] Yash P. Aneja and Kunhiraman P. K. Nair. “Bicriteria transportation problem”. In: *Management Science* 25.1 (1979), pp. 73–78. DOI: 10.1287/mnsc.25.1.73.
- [6] Carlos H. Antunes, Maria João Alves, and João Clímaco. *Multiobjective linear and integer programming*. EURO Advanced Tutorials on Operational Research. Cham: Springer, 2016. ISBN: 978-3-319-28744-7; 978-3-319-28746-1. DOI: 10.1007/978-3-319-28746-1.
- [7] Antonio Benítez-Hidalgo et al. “jMetalPy: A Python framework for multi-objective optimization with metaheuristics”. In: *Swarm and Evolutionary Computation* (2019), p. 100598. ISSN: 2210-6502. DOI: <https://doi.org/10.1016/j.swevo.2019.100598>. URL: <http://www.sciencedirect.com/science/article/pii/S2210650219301397>.
- [8] Harold P. Benson. “An outer approximation algorithm for generating all efficient extreme points in the outcome set of a multiple objective linear programming problem”. In: *Journal of Global Optimization* 13 (1998), pp. 1–24.
- [9] Julian Blank and Kalyanmoy Deb. “Pymoo: Multi-Objective Optimization in Python”. In: *IEEE Access* 8 (2020), pp. 89497–89509.
- [10] Fritz Bökler, Levin Nemesch, and Mirko H. Wagner. *PaMILO: A Solver for Multi-Objective Mixed Integer Linear Optimization and Beyond*. 2023. DOI: 10.48550/arXiv.2207.09155.
- [11] Ralf Borndörfer et al. “PolySCIP”. In: *Proceedings of the International Conference on Mathematical Software*. Ed. by G.-M. Greuel et al. Vol. 9725. Lecture Notes in Computer Science. 2016.
- [12] L.G. Chalmet, L. Lemonidis, and D.J. Elzinga. “An algorithm for the bi-criterion integer programming problem”. In: *European Journal of Operational Research* 25 (1986), pp. 292–300. DOI: 10.1016/0377-2217(86)90093-7.

- [13] COIN-OR projects. *Clp/Cbc*. Date visited: 2025-12-31. 2025. URL: <https://www.coin-or.org/>.
- [14] COPT GmbH. *COPT (Cardinal Optimizer)*. Date visited: 2025-12-31. 2025. URL: <https://www.copt.de/>.
- [15] Laszlo Csirmaz. “Inner approximation algorithm for solving linear multiobjective optimization problems”. In: *Optimization* 70 (7 2021), pp. 1487–1511.
- [16] Steven Diamond and Stephen Boyd. “CVXPY: A Python-embedded modeling language for convex optimization”. In: *The Journal of Machine Learning Research* 17.1 (2016), pp. 2909–2913.
- [17] Miguel Ángel Domínguez-Ríos, Francisco Chicano, and Enrique Alba. “Effective anytime algorithm for multiobjective combinatorial optimization problems”. In: *Information Sciences* 565 (2021), pp. 210–228.
- [18] Oscar Dowson, Xavier Gandibleux, and Gökhan Kof. *From vOptGeneric.jl to MultiObjectiveAlgorithms.jl*. Talk presented at JuliaDays 2023, 4-6 October 2023. Paris (France). 2023.
- [19] Oscar Dowson, Xavier Gandibleux, and Gökhan Kof. “MultiObjectiveAlgorithms.jl: a Julia package for solving multi-objective optimization problems”. In: *INFORMS Journal on Computing* (2026). To appear.
- [20] Iain Dunning, Joey Huchette, and Miles Lubin. “JuMP: A Modeling Language for Mathematical Optimization”. In: *SIAM Review* 59.2 (2017), pp. 295–320. DOI: 10.1137/15M1020575. URL: <https://doi.org/10.1137/15M1020575>.
- [21] Juan J. Durillo and Antonio J. Nebro. “jMetal: A Java framework for multi-objective optimization”. In: *Advances in Engineering Software* 42.10 (2011), pp. 760–771. ISSN: 0965-9978. DOI: <https://doi.org/10.1016/j.advengsoft.2011.05.014>. URL: <https://www.sciencedirect.com/science/article/pii/S0965997811001219>.
- [22] Matthias Ehrgott. *Multicriteria optimization*. 2nd ed. Vol. 491. Lecture Notes in Economics and Mathematical Systems. Berlin: Springer-Verlag, 2005.
- [23] FICO. *FICO Xpress Optimization*. Date visited: 2025-12-31. 2025. URL: <https://www.fico.com/en/products/fico-xpress-optimization>.
- [24] GAMS Development Corporation. *GAMS Documentation*. Date visited: 2025-12-31. 2025. URL: <https://www.gams.com/>.
- [25] Xavier Gandibleux, Gauthier Soleilhac, and Anthony Przybylski. *vOptSolver: an ecosystem for multi-objective linear optimization*. Talk presented at JuliaCon 2021, 28-30 July 2021 (online). 2021.
- [26] Xavier Gandibleux et al. *vOptSolver, a “get and run” solver of multiobjective linear optimization problems built on Julia and JuMP*. MCDM2017: 24th International Conference on Multiple Criteria Decision Making. July 10-14, 2017. Ottawa (Canada). 2017.
- [27] Xavier Gandibleux et al. *vOptSolver: an open source software environment for multiobjective mathematical optimization*. IFORS2017: 21st Conference of the International Federation of Operational Research Societies. July 17-21, 2017. Quebec City (Canada). 2017.

- [28] GNU project: Free Software Foundation, Inc. *GLPK: GNU Linear Programming Kit*. Date visited: 2025-12-31. 2025. URL: <https://www.gnu.org/software/glpk/>.
- [29] GNU project: Free Software Foundation, Inc. *GNU MathProg modeling language*. Date visited: 2025-12-31. 2025. URL: <https://www.gnu.org/software/glpk/>.
- [30] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*. Date visited: 2025-12-31. 2025. URL: <https://www.gurobi.com>.
- [31] David Hadka and Patrick Reed. “Diagnostic Assessment of Search Controls and Failure Modes in Many-Objective Evolutionary Optimization”. In: *Evolutionary Computation* 20.3 (Sept. 2012), pp. 423–452. ISSN: 1063-6560. DOI: 10.1162/EVCO_a_00053. eprint: https://direct.mit.edu/evco/article-pdf/20/3/423/1494624/evco_a_00053.pdf. URL: https://doi.org/10.1162/EVCO_a_00053.
- [32] Yacov Y. Haimes, Leon S. Lasdon, and David A. Wismer. “On a Bicriterion Formulation of the Problems of Integrated System Identification and System Optimization”. In: *IEEE Transactions on Systems, Man, and Cybernetics SMC-1.3* (1971), pp. 296–297.
- [33] William E. Hart, Jean-Paul Watson, and David L Woodruff. “Pyomo: modeling and solving mathematical programs in Python”. In: *Mathematical Programming Computation* 3.3 (2011), pp. 219–260. URL: <https://www.pyomo.org>.
- [34] Hexaly. *Hexaly Optimizer and Modeler*. Date visited: 2025-12-31. 2025. URL: <https://www.hexaly.com/>.
- [35] HiGHS. *HiGHS*. Date visited: 2025-12-31. 2025. URL: <https://highs.dev/>.
- [36] IBM. *IBM ILOG CPLEX Optimization Studio Documentation*. Date visited: 2025-12-31. 2025. URL: <https://www.ibm.com/products/ilog-cplex-optimization-studio>.
- [37] Heinz Isermann. “Linear lexicographic optimization”. In: *OR Spectrum* 4.4 (1982), pp. 223–228. ISSN: 0171-6468. DOI: 10.1007/BF01782758. URL: <https://doi.org/10.1007/BF01782758>.
- [38] Gokhan Kirlık and Serpil Sayın. “A new algorithm for generating all non-dominated solutions of multiobjective discrete optimization problems”. In: *European Journal of Operational Research* 232 (2014), pp. 479–488. DOI: 10.1016/j.ejor.2013.08.001.
- [39] Thorsten Koch. “The ZIMPL modeling language”. en. In: *Optima* 103 (2017), pp. 8–9. URL: <http://www.mathopt.org/Optima-Issues/optima103.pdf>.
- [40] Melissa Koenen, Marleen Balvert, and Hein Fleuren. *A Renewed Take on Weighted Sum in Sandwich Algorithms: Modification of the Criterion Space*. Tech. rep. CentER, Center for Economic Research, 2023.
- [41] Benoît Legat et al. “MathOptInterface: a data structure for mathematical optimization problems”. In: *INFORMS Journal on Computing* 34.2 (2021), pp. 672–689.

- [42] Andreas Löhne and Benjamin Weissing. “The vector linear program solver BENSOLVE – notes on theoretical background”. In: *European Journal of Operational Research* 260.3 (2017), pp. 807–813. ISSN: 0377-2217. DOI: <http://dx.doi.org/10.1016/j.ejor.2016.02.039>. URL: <http://www.sciencedirect.com/science/article/pii/S0377221716300765>.
- [43] Miles Lubin et al. “JuMP 1.0: Recent improvements to a modeling language for mathematical optimization”. In: *Mathematical Programming Computation* 15.3 (2023), pp. 581–589.
- [44] MATLAB. *paretosearch – Find points in Pareto set*. Date visited: 2025-12-31. 2025. URL: <https://au.mathworks.com/help/gads/paretosearch.html>.
- [45] Kaisa Miettinen. *Nonlinear Multiobjective Optimization*. Boston: Kluwer Academic Publishers, 1999.
- [46] Giovanni Misitano et al. “DESDEO: The Modular and Open Source Framework for Interactive Multiobjective Optimization”. In: *IEEE Access* 9 (2021), pp. 148277–148295. DOI: 10.1109/ACCESS.2021.3123825.
- [47] MOSEK. *MOSEK optimization software*. Date visited: 2025-12-31. 2025. URL: <https://www.mosek.com/>.
- [48] Antonio J. Nebro and Xavier Gandibleux. “Experiences Using Julia for Implementing Multi-objective Evolutionary Algorithms”. In: *Metaheuristics*. Ed. by Marc Sevaux et al. Cham: Springer Nature Switzerland, 2024, pp. 174–187. ISBN: 978-3-031-62922-8.
- [49] J.S. Roy. *PuLP 3.3.0*. Date visited: 2025-12-31. 2025. URL: <https://pypi.org/project/PuLP/>.
- [50] Ludovic Salomon, Daniel Dörfler, and Andreas Löhne. *MOCVXPY: a CVXPY extension for multiobjective optimization*. 2025. DOI: 10.48550/arXiv:2510.21010.
- [51] Ralph E. Steuer. “ADBASE: A Program for Analyzing Multiple Objective Linear Programming Problems”. In: *Journal of Marketing Research* 12 (1975), pp. 453–455.
- [52] Satya Tamby and Daniel Vanderpooten. “Enumeration of the Nondominated Set of Multiobjective Discrete Optimization Problems”. In: *INFORMS Journal on Computing* 33.1 (2021), pp. 72–85.
- [53] Milan Zeleny. *Linear Multiobjective Programming*. Lecture Notes in Economics and Mathematical Systems. Berlin, Heidelberg: Springer, 1974. DOI: 10.1007/978-3-642-80808-1. URL: <https://doi.org/10.1007/978-3-642-80808-1>.
- [54] Milan Zeleny. “Multicriteria Simplex Method: A Fortran Routine”. In: *Multiple Criteria Decision Making Kyoto 1975*. Ed. by Milan Zeleny. Vol. 123. Lecture Notes in Economics and Mathematical Systems. Berlin, Heidelberg: Springer, 1976.